

Build It From Scratch

How to rebuild the entire MET / NIMHANS prep system on a fresh Claude account, starting from zero.

The Notes-for-Exam book library · the mock & test engine · the live trackers · the offline pack.

Written as a build manual, not a chat log. June 2026.

This is the playbook. If you handed a stranger a brand-new Claude account, an empty GitHub, and the same pile of syllabus and past-paper PDFs, this document tells them exactly how to reproduce everything that's on the site. No prior code knowledge assumed. You will be telling Claude what to build; Claude writes the code. Your job is to know *what to ask for, in what order, and how to feed it the source material*.

What's inside

- | | | | |
|---|--------------------------------------|----|--|
| 0 | The mental model | 7 | Teaching to depth (the explanations) |
| 1 | One-time setup (the cold start) | 8 | The live trackers (admissions + news) |
| 2 | The skeleton site, live in an hour | 9 | The offline pack (PWA, APK, single file) |
| 3 | Feeding Claude the source PDFs | 10 | Polish: gate, fullscreen, sync |
| 4 | The Notes-for-Exam book library | 11 | The prompting playbook |
| 5 | The mock & test engine | 12 | Gotchas that will bite you |
| 6 | Turning a real PYQ paper into a mock | 13 | The build order, end to end |

0 • The mental model

Before any code, understand the shape of the thing, because every later decision falls out of it.

It is **one static website**. No server, no database, no backend, no login system, no monthly bill. It is a folder of HTML, CSS and JavaScript files sitting in a GitHub repository. GitHub serves that folder for free at a public URL (GitHub Pages). That single decision is what makes the whole thing cheap, fast, durable, and trivially offline-able.

Everything you see is a consequence of that:

- **Each mock test is a single self-contained HTML file.** The questions, the answers, the scoring logic, the explanations: all baked into one file. Open it, it runs. No network call.
- **"Your data" never leaves the phone.** Scores, notes, and progress are stored in the browser's own `localStorage`. There is nothing to hack, nothing to sync, no privacy surface.
- **"Offline" is almost free.** Because it's just files, you can wrap the whole folder in a Service Worker (a browser feature that caches files) and it keeps working with the WiFi off. The Android APK is literally that same folder in a thin wrapper.
- **The trackers update themselves.** A scheduled job (GitHub Actions) runs twice a day, asks for fresh data, rewrites a small JSON file, and commits it. The site re-reads that file. No human in the loop.

Why this matters for you: you are not "programming." You are directing. Claude can write a self-contained quiz HTML file in one shot if you describe it well. The skill you're building is *specification*, not coding. The rest of this manual is about how to specify each piece.

The toolchain, in one breath

Tool	What it's for	Cost
Claude (Pro/Max) + Claude Code	Writes all the code and content. Claude Code is the terminal app that can actually create and edit files on your machine, not just chat.	Subscription
GitHub account	Stores the files; hosts them live for free via Pages.	Free
Git	The "save and publish" mechanism. Claude runs it for you.	Free
A laptop with a terminal	Where Claude Code runs. Mac or Linux easiest; Windows works.	—
Python 3	Only for a few build scripts (reading PDFs, packing the APK). Claude writes these too.	Free

That's the whole stack. No cloud account, no hosting bill, no framework, no Node build step, no React. The deliberate primitiveness is the point: it will still work in five years.

1 • One-time setup (the cold start)

You're on a fresh machine with a fresh Claude account. Do these once.

1. **Get Claude.** Subscribe to Claude (Pro is fine to start; Max if you'll be doing this for hours daily). Install **Claude Code** from the official Anthropic instructions. It runs in your terminal. This is the piece that can create files, run git, and execute scripts; the web chat alone cannot.
2. **Make a GitHub account.** Free. Pick a username you don't mind being in the public URL (the site will live at <https://YOURNAME.github.io/REPO/>).
3. **Install git** and tell it who you are. Claude Code can walk you through this; on a Mac it's typically already there.
4. **Authenticate GitHub from the terminal** (the `gh` CLI is easiest). This lets Claude push your changes live without you pasting passwords each time.

The one sentence to open with. Once Claude Code is running in an empty folder, your very first message sets the contract for everything after. Use something like the prompt below. It tells Claude the architecture so it doesn't reach for a heavy framework.

OPENING PROMPT

"I'm building a static study-prep website for one exam. Hard constraints: plain HTML/CSS/vanilla JavaScript only, no frameworks, no build step, no backend. It will be hosted on GitHub Pages. Every test must be a single self-contained HTML file that works offline. All user data stays in localStorage. Set up an empty repo with a homepage and get it deploying to GitHub Pages so I can confirm the live URL works before we build anything real."

When Claude finishes, you should have a public URL that shows a near-empty page. **Confirm that URL loads in your phone's browser before doing anything else.** Everything from here is "add a page, push, refresh."

2 · The skeleton site, live in an hour

Don't build content yet. Build the frame the content will hang on. Ask Claude for:

- A **homepage** (`index.html`) with a top navigation bar: Dashboard, Tests, Notes, Results, Plan, Downloads. The links can point at empty pages for now.
- A **shared stylesheet** (`site.css`) defining a dark theme with CSS variables (background, panel, text, accent, good/bad/warning colours). Everything reuses this so the whole site looks like one app, not a pile of pages.
- A **folder per section**: `/tests/` , `/notes/` , `/results/` , each just an index page that lists what's inside.

Design choice that paid off: a single dark theme defined once in CSS variables, reused by every page including the self-contained quizzes. When a quiz is its own file, paste the same `:root` variable block at the top. It keeps 130+ pages visually identical without a framework.

Deploy hygiene (do this now, thank yourself later)

- Put an empty file named `.nojekyll` at the repo root. GitHub Pages otherwise tries to run a blog-builder (Jekyll) over your files and can silently drop folders that start with an underscore. The empty file turns that off.
- After *every* push, check the **Actions / Pages build status** in GitHub, not just that the push succeeded. A green push with a failed Pages build means your change isn't live. This trips everyone up once.

3 · Feeding Claude the source PDFs

This is the step people skip and then wonder why the content is generic. **The depth comes entirely from the source material you feed in.** Claude teaching "from memory" gives you a Wikipedia summary. Claude teaching *from the actual RCI syllabus PDF and the actual past papers* gives you something that matches the real exam. The difference is everything.

What to collect first

1. **The official syllabus PDF** for your exam. The real one, section by section, with the weightings.
2. **Every past-year question paper (PYQ) you can get**, ideally with an answer key. Even photographed or scanned ones.
3. **Any standard textbooks** the exam is based on (for this build: Pinel for biopsychology, Schultz for personality, Baron for social, etc.).
4. **Reputable mock papers** from coaching sources, clearly labelled as practice, not official.

How to hand them over

Claude Code reads files on your machine. Drop the PDFs in a folder and point at them. The trick is asking Claude to *extract verbatim first, interpret second*:

EXTRACTION PROMPT

"Here's the official syllabus PDF. Read it and give me back the exact section structure: every section, sub-section, and topic, with the mark weighting for each, word for word as printed. Do not summarise or reorganise it into your own categories yet. I want the skeleton to match the official document exactly."

Gotcha that recurs: PDFs lie. Scanned papers come out garbled when you pull plain text (columns interleave, ticks vanish). Two fixes that worked repeatedly: (1) for scanned/answer-key images, have Claude *vision-read* the page (look at it as an image) rather than parse the text layer; (2) for the question text itself, prefer the embedded text layer (PyMuPDF) when it's clean. Cross-check the count: if the paper says 100 questions and you extracted 100, good; if you got 92, find the 8 it dropped before trusting anything.

Build the skeleton to the official shape, not the forum shape. A real lesson here: an early version was structured around how students *talk about* the syllabus on forums. It got rejected. The rebuild followed the official syllabus PDF's own six-section structure exactly. Match the document, not the chatter.

4 • The Notes-for-Exam book library

This is the reading half of the system: the syllabus turned into actual study notes, presented as a shelf of "books" you can open and read like an e-reader.

4.1 Generate the notes, one section at a time

Don't ask for "notes on everything." Go section by section off the skeleton you extracted in Chapter 3. For each topic:

NOTES PROMPT (PER SECTION)

"Using the official syllabus section [X] and the textbook chapters I gave you, write exam-grade study notes for this section. Depth target: enough that someone reading only this could answer a hard MCQ on it. Use headings, short paragraphs, and call out the high-yield facts and the classic traps examiners set. Don't pad. Save it as a markdown file under content/[section-slug].md."

Keep each section in its own file. It makes later editing surgical: you can deepen one chapter without Claude regenerating the whole library.

4.2 Wrap the notes in a "book reader"

Raw markdown isn't pleasant to revise from on a phone. The reader presents each notes file as a paginated book: single-page in portrait, two-page spread in landscape, swipe to turn pages, theme-aware.

READER PROMPT

"Build a 'Notes for Exam' reader as a PWA at /notes-app/. It lists every notes file as a book on a shelf. Tapping one opens a paginated reader: one page in portrait, a two-page spread in landscape, both from the same code path using CSS column-width and a column count, not column-count:2. Pages turn by swiping. It must work offline. Generate it from the markdown files with a Python build script so I can rebuild after editing notes."

Two real reader bugs, so you skip them: (1) Page the spread by the article's measured content width (`clientWidth` -derived pitch), not the frame width, or deep pages drift out of the margins. (2) Do portrait single-page and landscape spread from *one* code path via column-width + a column-count variable; don't special-case with `column-count:2` . And whenever you edit the reader shell, bump the Service Worker cache name (see Ch. 9) or phones keep serving the old version.

4.3 Why a build script

The library is generated, not hand-written. A Python script (Claude writes it: `build_notes_app.py`) reads the markdown content files and emits the HTML pages, the shelf index, and the offline manifest. Edit content → re-run script → push. This is the pattern for anything with many similar pages.

5 • The mock & test engine

The testing half. The core insight: **one engine, many data files**. You build the quiz machinery once, then every mock is just a list of questions handed to the same machine.

5.1 What the engine does

- Shows one question at a time (or a scrollable list), records the chosen option.
- Scores on the real marking scheme (this build standardised on **+1 correct, -0.25 wrong**; pick yours and keep it consistent across every mock and the analysis page).
- On submit: shows the score, a section-by-section breakdown, bands, and the cut-off line.
- Lets the user type a note under any question and saves it.
- Persists the whole attempt to `localStorage` so "My Results" can list past attempts.

ENGINE PROMPT

"Build a self-contained quiz engine in one HTML file. It reads a questions array (each item: stem, four options, correct index, section tag, and an explanation). Features: one-question-at-a-time with a progress bar, flag-for-review, +1/-0.25 scoring, a results screen with per-section breakdown and a cut-off band, a per-question notes field, and full attempt history saved to localStorage. No network, no libraries. Make the questions array a clearly marked block at the top so I can drop new question sets in."

5.2 Every mock = a copy of the engine + a new question array

Once the engine file is solid, each new mock is: copy the file, replace the questions array, change the title. That's why there are two dozen mocks (Amity, IHBAS, RML, PYQ Real, the MET mocks, pep mocks, diagnostics) with zero duplicated effort. Claude does the copy-and-swap.

Keep the marking scheme sacred. Decide it once. If half your mocks score +4/-1 and half score +1/-0.25, the "My Results" comparison becomes meaningless. Standardise, and make the analysis page's scoring, max marks, bands, and cut-off all read from the same scale.

5.3 The diagnostic variant

One high-value variant: a diagnostic test that's weighted toward the section the student is weakest in, and whose *missed* questions auto-compile into a personalised study sheet at the bottom of the results. Same engine, plus a "collect the misses into a list" step on submit.

6 • Turning a real past paper into a mock

This is the highest-value, fiddliest task, so it gets its own chapter. Goal: take a real exam PDF and a real answer key and produce a faithful, attemptable mock, not a paraphrase.

The reliable recipe

1. **Render and read.** Have Claude open the PDF. If the text layer is clean, pull questions from it (PyMuPDF). If it's a scan, vision-read the pages.
2. **Extract the answer key separately.** Keys are often a grid of bold letters or ticked boxes. The bold-flag trick works: PyMuPDF exposes which characters are bold, so the correct option often falls out of "which letter is bold." For ticked scans, vision-read.
3. **Match key to questions by number,** not by position. Off-by-one in the key is the most common error; verify against a question you know.
4. **Count and reconcile.** If the paper has 100 Qs and the key covers only 70, say so out loud and mark the uncovered ones, don't invent answers.
5. **Flag the debatable ones.** Real papers have wrong or contested official keys. Keep the official answer but add a visible "△ key debate" note rather than silently overriding.
6. **Drop the junk honestly.** Duplicates, unreadable items, truly ambiguous ones: remove them and record how many you dropped and why. Don't pad back to a round number.

PYQ-TO-MOCK PROMPT

"Here's a real exam paper PDF and its answer key. Build a faithful untimed mock from it reusing my existing quiz engine. Extract questions verbatim, don't paraphrase. Pull the key by [bold-flag / vision-reading the ticks]. Match key to question by printed question number and verify against Q[n] which I know is [answer]. Tell me the final question count, exactly which items you dropped and why, and flag any answers where the official key looks contested. Don't fabricate answers for anything the key doesn't cover."

Real war stories from this build: one paper had corrupt compression and had to be repaired before it would render; pages 4–6 of another were rotated 90°; one file labelled as an "IHBAS PY paper" turned out to be a completely different exam (a UGC-NET paper) once read. Always actually read the source before trusting its filename.

7 · Teaching to depth (the explanations)

A mock that only tells you "correct answer: B" teaches nothing. The depth feature: **under every question, a full explanation, plus a line on why each wrong option is wrong.** That last part, the per-distractor traps, is what turns a test into a lesson.

The data shape

Extend each question with two fields: a main **explanation** (why the right answer is right, with the concept behind it) and a **traps** array, one short line per wrong option explaining the misconception that makes it tempting.

DEEP-EXPLANATION PROMPT

"For each question in this mock, write: (1) a concept-level explanation of why the correct answer is right, the kind a topper would give, not a one-liner; and (2) for each of the three wrong options, one line naming the specific misconception or look-alike fact that makes someone pick it. Output it as explanation + a traps array per question. Match the notes-style voice, bigger readable text, not a cramped tooltip."

How this scaled across 22 mocks: the reliable pattern was extract → hand each mock to a focused sub-task → write the explanations → port them into the engine's render → verify a sample by eye. Do it per mock, not all at once; quality drops when you ask for too much in one shot.

Companion pages that fall out of depth work

- **Look-alikes page:** a running list of confusable term-pairs ("tell these two apart"). Every time a trap is "they confused X with Y," X/Y becomes an entry.
- **Gap diagnostic / miss-breakdown:** feed in the exact questions a student got wrong and build a page that explains each miss and the underlying gap. (This is the "send me the 32 you missed and I'll break down every one" pattern.)

8 • The live trackers (admissions + news)

Two pages that have to stay current without you touching them: an **admissions tracker** (programs, deadlines, fees, seats) and a **news board** (exam notifications). The mechanism is the reusable part.

How self-updating works without a server

1. The page reads its data from a small **JSON file** in the repo (e.g. `news.json`), and also has the same data inlined as a fallback so it renders instantly.
2. A **GitHub Actions** workflow runs on a schedule (this build: 8am and 3pm IST). It runs a Python script (`refresh_trackers.py`) that fetches fresh info, rewrites the JSON, stamps a "last refreshed" time, and commits it back to the repo.
3. GitHub Pages redeploys automatically on that commit. Next time anyone opens the page, it's current.

TRACKER PROMPT

"Build an admissions tracker page that reads from data/admissions.json (programs, deadline, fee, seats, hostel, status). Render it as cards with a 'last refreshed' stamp at the top. Then add a GitHub Actions workflow that runs twice daily, runs a Python script to refresh the JSON, and commits the change so Pages redeploys. Keep an inlined copy of the JSON in the HTML as a fallback so the page never renders empty, and remember to re-sync that inlined copy whenever the script format changes."

The fallback trap: if you inline a copy of the data for instant render *and* load the JSON, you now have two copies. When you change one, change both, or the page shows stale inlined data to anyone whose fetch is slow. Cron times in Actions are UTC; convert from your local time carefully (8am IST is 2:30am UTC).

9 · The offline pack (PWA, APK, single file)

Three tiers of "works with no internet," from easiest to most packaged. You don't need all three; this build did all three because different people wanted different things.

Tier 1 · PWA + Service Worker (free, do this always)

A Service Worker is a bit of JavaScript the browser keeps running. It caches every file on first visit, so the site loads with the WiFi off and can be "Added to Home Screen" like an app. Add a `manifest.webmanifest` (app name, icons) and a `sw.js` that precaches the file list.

Two Service Worker rules learned the hard way: (1) **Bump the cache name on every deploy** (e.g. `met-prep-20260611-...`). The SW serves the cached version until the name changes; forget this and users see old content forever. A force-sync button and an auto-stamped cache name fix the "why is it stale" complaints. (2) **Never cache a non-200 response.** An early bug cached a 404 on a page that didn't exist yet (cache-first), so even after the page went live it kept 404ing. Use network-first for navigation and never store a non-200.

Tier 2 · The Android APK (sideloadable app)

The APK is the same site folder wrapped so Android treats it as an installed app with its own icon, fully offline. No Google Play, no Android Studio needed for this approach: a Python script (`build_apk.py`) packs the site into a template APK and signs it with a local keystore. Claude writes the script; you keep the keystore file.

APK PROMPT

"Write a Python build script that takes my whole site folder and packs it into an installable Android APK that runs the site fully offline in a WebView, with its own home-screen icon and name. Sign it with a local keystore (generate one if missing). I don't have Android Studio; use a template APK + repackaging approach. Make it rebuildable so I can regenerate the APK after the site changes."

To keep the APK current, the same scheduled-job idea applies: a daily Action rebuilds it off the live site so re-installing always gives the latest.

Tier 3 · The single-file standalone (give one page to an outsider)

Sometimes you want to hand someone *one* mock with no access to the rest of the site and no internet. The recipe: take a page, strip the shared gate/nav/Service-Worker hooks, inline everything into one HTML file, and add a "Save offline" button that downloads the page to itself as a Blob. One file, double-click, works forever, reveals nothing else.

10 · Polish: gate, fullscreen, sync

- **Casual gate.** A shared `gate.js` asks for a passphrase before showing any page (this build: a single shared word). It is *not* real security, it just keeps the site out of random hands and search results. Don't store anything sensitive behind it and don't pretend it's auth.
- **Fullscreen / focus mode.** A one-tap button injected on every page that hides browser chrome for distraction-free revision. Injected site-wide by a small script rather than pasted into each file.
- **Results export.** Let the student copy their result out as plain readable text (a modal they can select and copy is the primary path; a `.txt` download is the fallback). People want their scores in a form they can paste into a doc, not locked in the app.
- **Full backup.** One button that pulls every attempt + every note + the whole score history out of `localStorage` into a single plain-text file. `localStorage` lives only on that one device; without an export, a cleared browser wipes everything.

11 · The prompting playbook

The build is 10% architecture and 90% knowing how to ask. The patterns that actually worked:

Pattern	Why it works
Extract verbatim before you interpret.	Forces Claude to anchor on the real document instead of its own summary. Every depth win started here.
One section / one mock at a time.	Quality collapses when you ask for "all of it." Small, verifiable chunks stay accurate.
Always ask for the build script, not just the output.	You'll regenerate everything ten times. A script makes "edit content, rebuild, push" a 30-second loop.
Make Claude report what it dropped.	"Tell me the final count and exactly what you removed and why" catches silent fabrication and missing questions.
Verify against a known answer.	"Check this against Q12, which I know is C" instantly surfaces off-by-one key errors.
Reuse, don't reinvent.	"Build this the same way as the existing X" keeps 130 pages consistent and is far faster than describing from scratch.
State the hard constraints every time the context resets.	"Vanilla JS, no framework, single self-contained file" up front stops Claude reaching for React.

The meta-move: after each working piece, ask Claude to write down *how* it built it (the recipe, the gotchas) in a notes file. Next time you build a similar thing, hand it that note. The build accelerates as the recipe library grows; that's exactly how this very manual exists.

12 · Gotchas that will bite you

Concentrated list of every trap this build hit, so you don't pay for them twice.

Symptom	Cause & fix
Push succeeds but the site doesn't change.	Pages build failed silently. Add <code>.nojekyll</code> ; always check the Pages build status, not just the push.
Live page keeps 404ing after you add it.	Service Worker cached the 404 (cache-first). Use network-first for navigation; never cache a non-200.
Users see old content after an update.	SW cache name unchanged. Auto-stamp the cache name per build; add a force-sync button.
Extracted PYQ has fewer questions than the paper.	Text-layer dropped some (columns, rotation, corrupt compression). Count and reconcile; vision-read the hard pages; repair/rotate first.
Answer key is one option off everywhere.	Matched key by position, not printed number. Re-match by question number; verify against a known answer.
Tracker shows stale data instantly then updates.	Inlined fallback copy out of sync with the JSON. Re-sync the inlined copy whenever the data changes.
Scheduled job runs at the wrong time.	GitHub Actions cron is UTC. Convert from your timezone (IST = UTC+5:30).
git push hangs or dies on a synced folder.	Cloud-synced folders (iCloud/Dropbox) deadlock git's memory-mapping. Work from a local <code>/tmp</code> clone and push from there.
Book reader pages drift out of the margins on deep pages.	Paged by frame width instead of measured content pitch. Use the article's <code>clientWidth</code> -derived pitch.

13 · The build order, end to end

If you did nothing but follow this list top to bottom, you'd reproduce the whole system.

1. **Setup** (Ch. 1): Claude Code, GitHub, git authenticated. Open with the architecture-setting prompt.
2. **Skeleton** (Ch. 2): homepage, shared dark stylesheet, empty section folders, deploying live with `.nojekyll`. Confirm the URL loads on your phone.
3. **Collect sources** (Ch. 3): syllabus PDF, every PYQ + key, textbooks. Extract the official syllabus structure verbatim.
4. **Notes library** (Ch. 4): section-by-section notes → build script → book reader PWA.
5. **Quiz engine** (Ch. 5): one self-contained engine file with localStorage, +1/−0.25 scoring, per-section results.
6. **Real PYQ mocks** (Ch. 6): turn each past paper into a faithful mock via the extract-and-reconcile recipe.
7. **Depth pass** (Ch. 7): add full explanations + per-distractor traps to each mock; spin off the look-alikes and miss-breakdown pages.
8. **Trackers** (Ch. 8): admissions + news pages reading JSON, with a twice-daily GitHub Action keeping them fresh.
9. **Offline pack** (Ch. 9): Service Worker/PWA first (always), then the APK build script, then single-file standalones for giveaways.
10. **Polish** (Ch. 10): gate, fullscreen, results export, full backup.
11. **Maintain**: as new papers and notifications arrive, it's just "add a page, rebuild, push." The system is designed so additions are cheap forever.

The honest bottom line. None of this is hard in the sense of clever code. It's hard in the sense of *discipline*: feed real sources, work in small verifiable chunks, make everything rebuildable, and write down the recipe each time. Do that with a fresh Claude account and you end up exactly here: a fast, free, offline-capable prep system that out-teaches anything you could buy, because it's built from the actual exam, not a summary of it.